HDOC Day-58

Kth Smallest Element Using Bubble Sort and Pointers

Aim

Write a C program using a variable-length integer array and pointers to sort the array using Bubble Sort and display the kth smallest element, counting duplicates separately. Display -1 for invalid k.

Algorithm

1. Start and read a positive array size.
2. Declare a variable-length integer array and read its elements.
3. Read integer k and pass the array address, size, and k to kth_smallest().
4. If k is not between 1 and size inclusive, display -1 and return.
5. Use nested loops to bubble-sort the array into ascending order by swapping adjacent out-of-order elements.
6. Print the element at index k-1 (duplicates remain separate positions).
7. Stop.

Test Case Table

Test Case	Array (size = number of values)	k	Expected Output	Purpose
1	7 10 4 3 20 15	3	7	Standard example
2	7 10 4 3 20 15	4	10	Standard example
3	7 10 4 3 20 15	7	-1	k exceeds size
4	4 2 2 7 5	3	4	Duplicate elements counted separately
5	4 2 2 7 5	2	2	Duplicate at kth position
6	-5 -1 -3 2	2	-3	Negative numbers
7	9	1	9	Single element
8	5 1 4 2 3	0	-1	k is zero
9	5 1 4 2 3	5	5	k equals size
10	8 8 8 8	3	8	All duplicates

Expected Output is the number printed after "kth smallest element:"; for invalid k the program prints only -1.

C Program

```
#include <stdio.h>

void kth_smallest(int *arr, int size, int k)
{
    int i, j, temp;
    if (k <= 0 || k > size) {
        printf("-1\n");
        return;
    }
    for (i = 0; i < size - 1; i++) {
        for (j = 0; j < size - i - 1; j++) {
            if (*(arr + j) > *(arr + j + 1)) {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
```

```
    }
    printf("kth smallest element: %d\n", *(arr + k - 1));
}

int main(void)
{
    int size, k, i;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 10000) {
        printf("Invalid size. Enter a positive integer up to 10000.\n");
        return 1;
    }
    int arr[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++) {
        if (scanf("%d", arr + i) != 1) {
            printf("Invalid array element.\n");
            return 1;
        }
    }
    printf("Enter k: ");
    if (scanf("%d", &k) != 1) {
        printf("Invalid k.\n");
        return 1;
    }
    kth_smallest(arr, size, k);
    return 0;
}
```

Compilation and Execution Commands

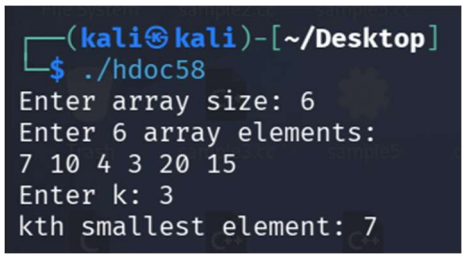
gcc hdoc58.c -o hdoc58
./hdoc58

Compilation and Execution



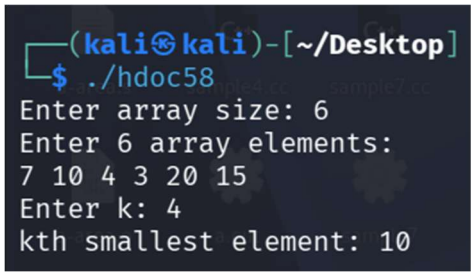
Test Case 1

Enter size = 6; elements = 7 10 4 3 20 15; k = 3. Expected result: 7. (Standard example)



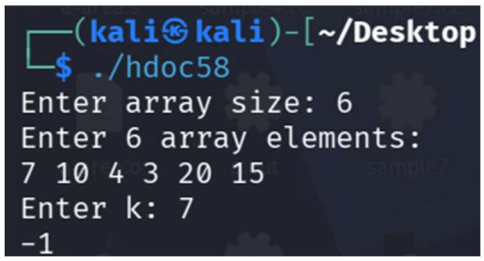
Test Case 2

Enter size = 6; elements = 7 10 4 3 20 15; k = 4. Expected result: 10. (Standard example)



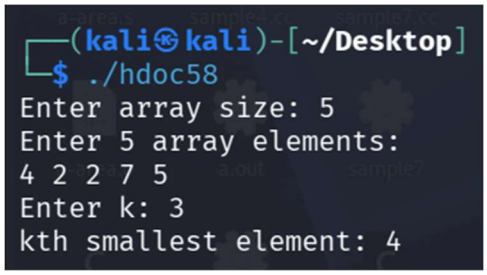
Test Case 3

Enter size = 6; elements = 7 10 4 3 20 15; k = 7. Expected result: -1. (k exceeds size)



Test Case 4

Enter size = 5; elements = 4 2 2 7 5; k = 3. Expected result: 4. (Duplicate elements counted separately)



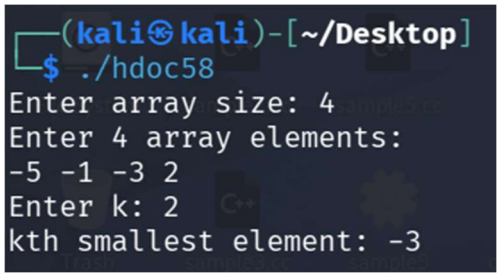
Test Case 5

Enter size = 5; elements = 4 2 2 7 5; k = 2. Expected result: 2. (Duplicate at kth position)



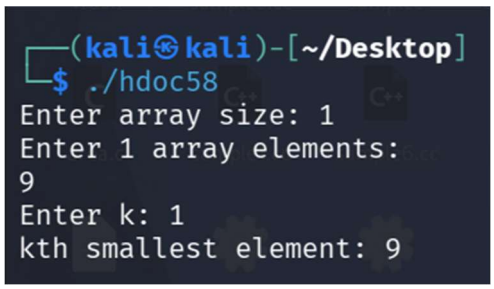
Test Case 6

Enter size = 4; elements = -5 -1 -3 2; k = 2. Expected result: -3. (Negative numbers)



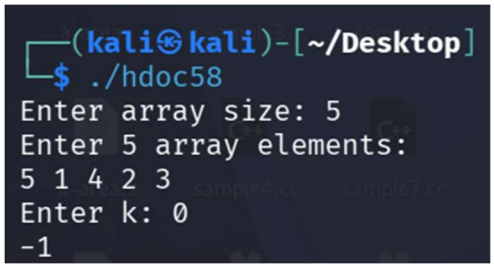
Test Case 7

Enter size = 1; elements = 9; k = 1. Expected result: 9. (Single element)



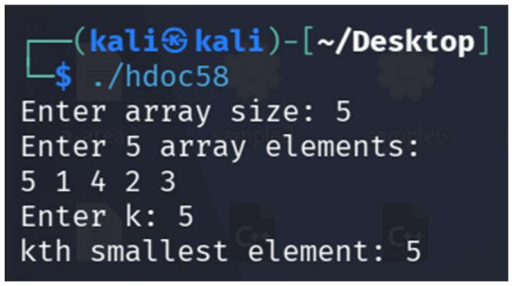
Test Case 8

Enter size = 5; elements = 5 1 4 2 3; k = 0. Expected result: -1. (k is zero)



Test Case 9

Enter size = 5; elements = 5 1 4 2 3; k = 5. Expected result: 5. (k equals size)



Test Case 10

Enter size = 4; elements = 8 8 8 8; k = 3. Expected result: 8. (All duplicates)

